

Implementation of the SIMPLE Algorithm using FVM for 2D Steady Incompressible N-S Equations

In this document, I will be going over the implementation of the Semi-Implicit Method for Pressure Linked Equations (SIMPLE).

This document can be used to understand the coded SIMPLE algorithm implementation in the accompanying [Github repo](#).

Honestly, this is more for me so that I can keep track of the implementation for debugging (which helped a lot since I referred to this often to resolve my errors).

Regardless, let's begin!

Table of Contents

Governing Equations	2
SIMPLE Algorithm	2
Pressure Correction	4
Solution Update	6
Implementing SIMPLE Algorithm for FVM	6
Pressure Correction	7
Boundary Condition	10
Solution Update	12
Boundary Condition	13
Conclusion.....	13

Governing Equations

The governing equations for our problem are the continuity equation (i.e. conservation of mass) and the steady incompressible N-S equations (i.e. conservation of momentum). They have been written down in Equation (1) and Equation (2) in vector notation. If you are unsure of how we got here, which is completely fine, I recommend researching how these equations came to be.

$$\text{Continuity: } \nabla \cdot \mathbf{U} = 0 \quad (1)$$

$$\text{Incompressible N-S: } (\mathbf{U} \cdot \nabla) \mathbf{U} - \nabla \cdot (\nu \nabla \mathbf{U}) = -\nabla p \quad (2)$$

$$\text{where } \mathbf{U} = [u, v]^T$$

Notice that there are 3 equations and 3 unknowns. For the first term on the LHS of Equation (2), we will refer to this as the convection term, as it represents the transport of momentum by the flow. For the second term on the LHS of Equation (2), we will refer to this as the diffusive term, as it represents the diffusion of momentum.

Also notice that there is *no explicit equation for the evolution of pressure*. Rather we have two equations for the velocity components and a constraint that comes from the continuity equation.

In order to solve this system of PDEs we will be implementing the SIMPLE algorithm. This is an iterative solution procedure where the algorithm solves the momentum equations to predict the velocity then uses continuity to obtain a pressure correction. The pressure and velocity fields are accordingly updated and the process is repeated until solution convergence. Don't worry! This will all be explained in the following sections.

SIMPLE Algorithm

The starting point for the implementation of the algorithm is the finite-volume discretization of the incompressible Navier–Stokes momentum equation, which can be written in algebraic form as Equation (3).

$$\mathcal{M}U = -\nabla p \quad (3)$$

I will not be going over the finite volume formulation to build the $\mathcal{M}$ matrix but let us discuss what $\mathcal{M}$ represents. It is a discrete spatial operator that represents the discretized momentum equations for the finite volume formulation capturing the contributions of the convection and diffusion. Maybe simpler to think about is that when we matrix multiply $\mathcal{M}$ with U (a vector of the velocity at the cell centers), each row of $\mathcal{M}U$ is the incompressible

N-S equations for a respective cell. The right-hand side is simply the pressure gradient term i.e. the pressure contribution to momentum.

So, with a starting point of Equation (3), we begin the SIMPLE algorithm by decomposing $\mathcal{M}$ into a diagonal and off-diagonal component as in Equation (4).

$$\mathcal{M}U = \mathcal{A}U - \mathcal{H} = -\nabla p \quad (4)$$

where $\mathcal{A} \rightarrow$ contains the diagonal terms of $\mathcal{M}$

$\mathcal{H} \rightarrow$ contains the off-diagonal terms of $\mathcal{M}$

Okay, so the treatment of these terms is where the *semi-implicit* part of the name comes in. This is because for $\mathcal{H}$ and $-\nabla p$, we are going to evaluate them explicitly with either the previous iteration or initial solution (if it's the first iteration). Now when solving for U in $\mathcal{A}U$, the U represents the next unknown state velocities that we solve for, so that term is being treated implicitly. Maybe a better way to show this is as below in Equation (5) where you can see the explicit terms using the previous already known velocity and pressure and the implicit term using the future state (i.e. solution).

$$\mathcal{A}(U^{(k)})U^* - \mathcal{H}(U^{(k)}) = -\nabla p^{(k)} \quad (5)$$

where $U^{(k)} \rightarrow$ velocity from previous iteration

$p^{(k)} \rightarrow$ pressure from previous iteration

$U^* \rightarrow$ new velocity being solved for (currently unknown)

It is important to note that we do this in order to linearize the equation. As can be seen in Equation (5) U^* is multiplied by the discrete operator which does not depend on U^* itself making it linear. If we instead used $\mathcal{A}(U^*)$ then the solution would have been non-linear. This linearization greatly simplifies the solving process since this allows us to use some iterative method to get to the correct solution by solving a linear equation at each iteration.

Note also that $\mathcal{A}$ is a diagonal matrix which is very easy to invert (the inverse is just one over the diagonal components). This will become useful in later steps where the inverse of $\mathcal{A}$ (i.e. $\mathcal{A}^{-1}$) will be needed for solving the pressure and velocity.

At this point we are already (kind of) at the halfway point. We can evaluate the explicit terms in order to solve for U^* . The issue is that the U^* we solve for is based on just the linearized momentum equation. There is no saying that U^* satisfies the continuity equation which is another equation we need to satisfy. As such, note that U^* is this intermediate solution where we have only considered the momentum equations.

So how can we solve for U^{new} in a manner that is consistent with the momentum equations *and* the continuity equation? The answer is by changing the pressure, our other degree of freedom (per say), to enforce continuity then making sure that the momentum equation is still satisfied by altering U^* according to the pressure that was solved for based on the momentum equations. That was a mouthful. Let's take it slower by considering the first step...

Pressure Correction

This first step, often called the pressure correction step, is basically solving for a change in the pressure field in order to enforce continuity. Why solve for a change to the current pressure field instead of just solving for the pressure field? Might be a dumb question, but it is one I asked. The answer is that they are the same thing. Consider Equation (6) below.

$$p^{new} = p^{(k)} + p' \quad (6)$$

where $p' \rightarrow$ correction to the pressure

Whether we solve for p^{new} or p' they were both determined by the same linearized state so for a linear system they will always just have a difference of p^k . So, let's solve for p' since later on we are going to scale the change by an under-relaxation factor anyway (so we end up using p' anyway regardless of if we get it from $p^{new} - p^{(k)}$ or just directly.

We can solve for p_{new} by considering the equation for U^* and U^{new} (our new continuity satisfying velocity field) as in Equation (7) and (8), respectively, coming from rearranging Equation (5).

$$U^* = \mathcal{A}^{-1}(U^{(k)}) (\mathcal{H}(U^{(k)}) - \nabla p^{(k)}) \quad (7)$$

$$U^{new} = \mathcal{A}^{-1}(U^{(k)}) (\mathcal{H}(U^{(k)}) - \nabla p^{new}) \quad (8)$$

Every term on the RHS of Equation (7) is explicit, but in Equation (8) we decide to use an unknown p^{new} . Why are we changing the pressure? Why not one of the $U^{(k)}$ in the explicit terms? Well because that would make Equation (7) non-linear and implicit which we want to avoid for simplicity (and overall ability to solve). Also, just as a thought experiment, consider what it would be like if we kept pressure explicit and turned the other explicit terms implicit as in Equation (8)

$$U^{new} = \mathcal{A}^{-1}(U^{new}) (\mathcal{H}(U^{new}) - \nabla p^{(k)}) \quad (8)$$

(the above is just for a thought experiment, it's not justified (per say))

We would essentially be solving for a new velocity field that satisfies the pressure field from the previous iteration. We would basically be fixing the problem to the initial pressure

leaving ourselves with no mechanism to enforce continuity (i.e. more equations than unknowns, overdetermined problem). If you solve this, you would have just solved for a convection-diffusion equation with a fixed source term, not conservation of mass *and* momentum. It would be a one equation, one unknown problem taking away any coupling between pressure and velocity.

Okay, going back to the main derivation... importantly, note that Equation (7) and (8) are derived from the momentum equations and thus satisfy them. We now want to take the difference between the two equations in order to get the pressure correction term. The following math reflects said steps.

$$U^{new} - U^* = (\mathcal{A}^{-1}\mathcal{H} - \mathcal{A}^{-1}\nabla p^{new}) - (\mathcal{A}^{-1}\mathcal{H} - \mathcal{A}^{-1}\nabla p^*) \quad (9)$$

$$U^{new} - U^* = -\mathcal{A}^{-1}\nabla p^{new} + \mathcal{A}^{-1}\nabla p^* \quad (10)$$

$$U^{new} = U^* - \mathcal{A}^{-1}\nabla p^{new} + \mathcal{A}^{-1}\nabla p^* \quad (11)$$

$$U^{new} = U^* - \mathcal{A}^{-1}(\nabla p^{new} - \nabla p^*) \quad (12)$$

$$U^{new} = U^* - \mathcal{A}^{-1}\nabla p' \quad (13)$$

Now applying the continuity equation to Equation (13) we get a pressure Poisson equation for the pressure correction (p') as in Equation (15).

$$\nabla \cdot U^{new} = \nabla \cdot U^* - \nabla \cdot (\mathcal{A}^{-1}\nabla p') = 0 \quad (14)$$

$$\nabla \cdot (\mathcal{A}^{-1}\nabla p') = \nabla \cdot U^* \quad (15)$$

What we are doing when we solve Equation (14) for p' is finding a change to the pressure field that makes the new velocity, which satisfies the momentum equation, also satisfy the continuity equation.

So, everything looks perfect at this point. We have p' and we can solve for U^{new} which satisfies the momentum equation considering the new pressure field (p^{new}) and also satisfies continuity. So that's the right answer right? That's the end? NOPE.

Remember Equation (5), the one from everything came from. We treated a bunch of terms explicitly (hence, the semi-implicit) and linearized the problem. We still enforced continuity for U^{new} but now the momentum equation is not satisfied since

$$\mathcal{A}(U^{(k)}) \neq \mathcal{A}(U^{new}), \quad \mathcal{H}(U^{(k)}) \neq \mathcal{H}(U^{new}), \quad p^{(k)} \neq p^{new} \quad (16)$$

This might seem obvious for those familiar with numerical methods, but I think it is good to point back to exactly what has broken down due (so to say) in our linear equation.

Solution Update

So, our new solution is not going to satisfy momentum, but it is going to be an improvement compared to before. Why so? Because we stepped in the right direction. While the linearized approximation is not exact, it is sufficiently accurate locally that taking a step in the direction it predicts brings us closer to the true solution.

We do however want to add an under-relaxation factor since our linearization is only accurate locally. The more change we apply, the worse the linearization approximation. So, the solution update looks like Equation (17) and (18) for the pressure and velocity, respectively.

$$p^{(k+1)} = p^{(k)} + \alpha_p p' \quad (17)$$

where $\alpha_p \rightarrow$ under relaxation value for pressure correction (typical value: 0.3)

$$U^{(k+1)} = U^{(k)} + \alpha_U \left((U^* - \mathcal{A}^{-1} \nabla p') - U^{(k)} \right) \quad (18)$$

$\alpha_U \rightarrow$ under relaxation value for velocity correction (typical value: 0.7)

What we do from here is move repeat the cycle by using our solution that we solved for as those used in the explicit term for the next iteration which would look something like Equation (19) and Equation (20).

$$p^{(k)} = p^{(k+1)} \quad (19)$$

$$U^{(k)} = U^{(k+1)} \quad (20)$$

Then we loop through the same steps over and over until we finally reach a solution with a satisfactorily small residual for the continuity and momentum equations. That's it! That's the SIMPLE algorithm! Now how do we implement this in practice to a finite volume method scheme is something I wanted to discuss a little bit about.

Implementing SIMPLE Algorithm for FVM

I think it is easy to get lost in the math at times. Even if all the equations in the previous sections made sense, we can't really use them as is to write our own solver. The math in vector notation is not the end goal. Let me explain.

Considering that we are using the Finite Volume Method (FVM) on a collocated grid, we have our cells and our solution for the pressure and velocity would be at the cell centers. That is all the information we have. So, any sort of equation that we (or more so the computers) are going to solve has to be in terms of the cell center values.

So, for instance, consider Equation (18). We have a $\nabla p'$ term. Considering that we have the value of p' at the cell centers (since we solve for it), we would then have to express $\nabla p'$ using cell centered values. How can we do that? Well, that's what this section is for. Let's go through implementation of the steps for FVM starting with the ...

Pressure Correction

Recall that for the pressure correction, we are solving the formulate pressure Poisson equation as in Equation (15).

$$\nabla \cdot (\mathcal{A}^{-1} \nabla p') = \nabla \cdot U^* \quad (15)$$

For a finite volume formulation, the first step to be taken is taking the integral over the volume (or in this 2D case, the area). By doing so, we can get Equation (16).

$$\int_A \nabla \cdot (\mathcal{A}^{-1} \nabla p') dA = \int_A \nabla \cdot U^* dA \quad (21)$$

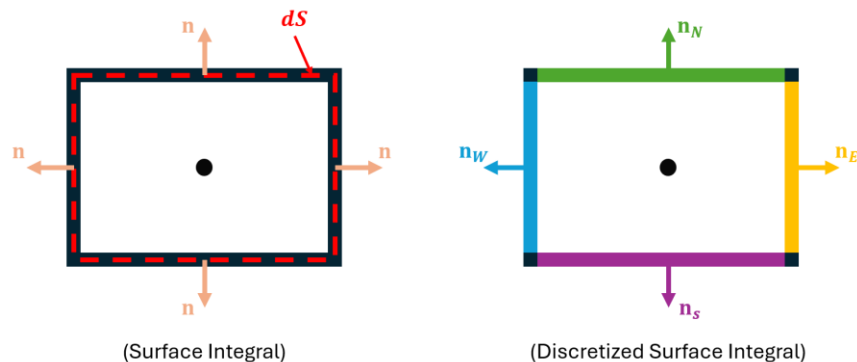
Notice that this is perfectly in a form that allows us to use divergence theorem on both sides. The reason for doing that is we no longer have to deal with the dot product once we apply divergence theorem. The general form of the divergence theorem is first shown for a vector field F over a domain Ω as in Equation (16).

$$\int_{\Omega} \nabla \cdot F dV = \oint_{d\Omega} F \cdot \mathbf{n} dS \quad (22)$$

Now applying the divergence theorem to Equation (21) we get the following Equation (23).

$$\oint_{dA} (\mathcal{A}^{-1} \nabla p') \cdot \mathbf{n} dA = \oint_{dA} U^* \cdot \mathbf{n} dA \quad (23)$$

For our finite volume cell, we can actually decompose this surface integral since we know our boundaries (they are the cell faces). Refer to the image below to see a visual representation of this explanation for a quadrilateral cell.



where $\mathbf{n}_N \rightarrow$ normal vector on north face

$\mathbf{n}_E \rightarrow$ normal vector on east face

$\mathbf{n}_S \rightarrow$ normal vector on south face

$\mathbf{n}_W \rightarrow$ normal vector on west face

Mathematically, this allows us to split the integral into four terms with each representing the contribution of one boundary. This would look like the following for the LHS term and RHS term from Equation (23) as in Equation (25) and (27), respectively.

$$\oint_{dA} (\mathcal{A}^{-1} \nabla p') \cdot \mathbf{n} dA = \sum_f^{\text{faces}} (\mathcal{A}^{-1} \nabla p') \cdot \mathbf{n}_f S_f$$

where $\mathbf{n}_f \rightarrow$ normal vector at face f

$S_f \rightarrow$ area of face f

$$\sum_f^{\text{faces}} (\mathcal{A}^{-1} \nabla p') \cdot \mathbf{n}_f S_f = (\mathcal{A}^{-1} \nabla p')_N \cdot \mathbf{n}_N S_N + (\mathcal{A}^{-1} \nabla p')_E \cdot \mathbf{n}_E S_E + (\mathcal{A}^{-1} \nabla p')_S \cdot \mathbf{n}_S S_S + (\mathcal{A}^{-1} \nabla p')_W \cdot \mathbf{n}_W S_W \quad (25)$$

$$\oint_{dA} (U^*) \cdot \mathbf{n} dA = \sum_f^{\text{faces}} (U^*) \cdot \mathbf{n}_f S_f$$

$$\sum_f^{\text{faces}} (U^*) \cdot \mathbf{n}_f S_f = (U^*)_N \cdot \mathbf{n}_N S_N + (U^*)_E \cdot \mathbf{n}_E S_E + (U^*)_S \cdot \mathbf{n}_S S_S + (U^*)_W \cdot \mathbf{n}_W S_W \quad (27)$$

Here we have decomposed the surface integral into a summation over faces which is a step towards the finite volume formulation but not the end. Recall that we want to express our solution in terms of cell center values because that is all we have. Right now, we have face values, so we need to rewrite this in terms of cell centers.

For $\mathcal{A}^{-1}$ from Equation (25) and U^* in Equation (27), we can use a linear interpolation between the current cell (cell P) and the neighboring cell (cell N) in order to get the face value. The general equation for such linear interpolation is as below in Equation (30).

$$d_{Pf} = \sqrt{(x_P - x_f)^2 + (y_P - y_f)^2} \quad (28)$$

$$d_{Nf} = \sqrt{(x_N - x_f)^2 + (y_N - y_f)^2} \quad (29)$$

where $\rightarrow x_f \equiv x$ -location of face centroid

$y_f \equiv y$ -location of face centroid

$$(var)_f = \frac{d_{Nf}(var)_P + d_{Pf}(var)_N}{d_{Nf} + d_{Pf}} \quad (30)$$

where $\rightarrow var \equiv$ variable of interest we are interpolating

We can simply replace var with the either $\mathcal{A}^{-1}$ or U^* to get the value at the face center from values at the cell centers through linear interpolation.

Now what about the $\nabla p'$ gradient term. For the gradient term, we can perform a finite difference between the current cell (cell P) and neighboring cell (cell N) as in Equation (31).

$$(\nabla p') \cdot \mathbf{n}_f = \frac{p'_N - p'_P}{d_{PN}} \quad (31)$$

Note that this is imperfect since it is a local gradient approximation and assumes that the vector from cell P to cell N is aligned with that of the face normal. This is where non-orthogonal correctors come into place but let us leave that for another document in the future.

Substituting Equation (30) and (31) into Equation (24) we get the following Equation (32) representing the LHS of Equation (23).

$$\sum_f^{\text{faces}} (\mathcal{A}^{-1} \nabla p') \cdot \mathbf{n}_f S_f = \sum_f^{\text{faces}} \left(\frac{d_{Nf}(\mathcal{A}^{-1})_P + d_{Pf}(\mathcal{A}^{-1})_N}{d_{Nf} + d_{Pf}} \right) \frac{p'_N - p'_P}{d_{PN}} S_f \quad (32)$$

For the RHS of Equation (23), which we decomposed to Equation (26), that can be much more easily written since there are no gradients, just the face value. So, substituting the interpolation we get Equation (33) below.

$$\sum_f^{\text{faces}} (U^*) \cdot \mathbf{n}_f S_f = \sum_f^{\text{faces}} \left(\frac{d_{Nf}(U^*)_P + d_{Pf}(U^*)_N}{d_{Nf} + d_{Pf}} \right) \cdot \mathbf{n}_f S_f \quad (33)$$

Recall that U^* is a two-component vector which has the x - and y -component of velocity. Therefore, we need to expand with the dot product accordingly.

$$\sum_f^{\text{faces}} \left(\frac{d_{Nf}(U^*)_P + d_{Pf}(U^*)_N}{d_{Nf} + d_{Pf}} \right) \cdot \mathbf{n}_f S_f = \left(\frac{d_{Nf}(u^*)_P + d_{Pf}(u^*)_N}{d_{Nf} + d_{Pf}} \right) \mathbf{n}_{fx} + \left(\frac{d_{Nf}(v^*)_P + d_{Pf}(v^*)_N}{d_{Nf} + d_{Pf}} \right) \mathbf{n}_{fy} \quad (34)$$

We can represent both of these equations in a matrix form by rewriting the equations as Equation (35).

$$\mathcal{L}p' = b \quad (35)$$

where $p' \rightarrow$ vector of pressure correction values that we are solving for

$\mathcal{L} \rightarrow$ matrix that applies the operation in Equation (32) to p'

$b \rightarrow$ vector of explicit evaluations of Equation (33)

Then all that is needed is a matrix solve in order to get the pressure correction values. Simple enough, but don't forget that with any PDE there is always that all important...

Boundary Condition

Let us consider three different types of boundary conditions that were used in the accompanying code. Note that this pressure Poisson PDE has a spatial second-order derivative for the pressure correction term (p') and a spatial first-order derivative for the intermediate velocity (U^*). So, what values of the pressure correction do we want to enforce at the boundaries? Let us answer this question starting with the boundary condition for the...

Inlet

At the velocity inlet, we have a prescribed (i.e. Dirichlet boundary condition) velocity and the pressure is a zero-gradient pressure (i.e. Homogeneous Neuman boundary condition).

For the pressure correction “flux” (i.e. $\mathcal{A}^{-1} \nabla p' \cdot \mathbf{n}_f S_f$) we want a zero-gradient for the pressure. To maintain the zero-gradient where the pressure at the inlet face (p_f) is equal to that of the cell center (p_p), any change in the pressure at the cell center must be equal to a change in the inlet face pressure (i.e. $p'_p = p'_f$). Consider what the pressure correction flux at the inlet face would be as shown in Equation (36).

$$\text{pressure correction flux @ inlet face} \equiv (\mathcal{A}_f^{-1}) \frac{p'_f - p'_p}{d_{pf}} S_f = 0 \quad (36)$$

Since $p'_p = p'_f$, the entire term is equal to zero, therefore, we need not make any contribution to the flux for the pressure correction term through the inlet face.

For the intermediate velocity “flux” (i.e. $(U^*) \cdot \mathbf{n}_f S_f$), we already know the values at the face so we can just substitute them as in Equation (38) below.

$$\text{intermediate velocity flux @ inlet face} \equiv (U^*) \cdot \mathbf{n}_{f,inlet} S_{f,inlet} \quad (37)$$

$$(U^*) \cdot \mathbf{n}_{f,inlet} S_{f,inlet} = \left(u^{inlet} n_{x_{f,inlet}} + v^{inlet} n_{y_{f,inlet}} \right) S_{f,inlet} \quad (38)$$

where $u^{inlet}, v^{inlet} \rightarrow$ prescribed inlet velocity

The values at the face for U^* will hold the prescribed inlet values (which is what we are using). By doing this, we are using the correct boundary flux for the intermediate velocity for the matrix solve. Now next, for the ...

Outlet

Same idea in that we need to consider the pressure Poisson term and the intermediate velocity term. Now for the outlet, we do not have a prescribed velocity, but we do have a prescribed pressure.

What we want to do here is enforce a pressure correction of zero, whereas for the inlet we had enforce a pressure correction gradient of zero. We don't want a correction of the pressure at the outlet so substituting $p'_f = 0$ we can arrive at Equation (39).

$$\text{pressure correction flux @ outlet face} \equiv (\mathcal{A}_f^{-1}) \frac{p'_f - p'_P}{d_{Pf}} S_f = (\mathcal{A}_f^{-1}) \frac{-p'_P}{d_{Pf}} S_f \quad (39)$$

Similar to the inlet, we prescribe the correct velocity for a zero-gradient velocity. That is done by making the velocity at the outlet face equal to that of the center of the cell touching the outlet. Therefore, we can just use the U^* values since those have the zero-velocity gradient values for velocity which would look as in Equation (41).

$$\text{intermediate velocity flux @ outlet face} \equiv (U^*) \cdot \mathbf{n}_{f,\text{inlet}} S_{f,\text{inlet}} \quad (40)$$

$$(U^*) \cdot \mathbf{n}_{f,\text{outlet}} S_{f,\text{outlet}} = (u_P^* n_{x_{f,\text{outlet}}} + v_P^* n_{y_{f,\text{outlet}}}) S_{f,\text{outlet}} \quad (41)$$

Okay now the inlet and outlet were pretty straightforward as was displayed. Now the last boundary we need to deal with is the...

Slip Wall

The slip wall is actually pretty simple. Just as with the inlet and the outlet, we already prescribed the correct pressure at the starting point of $\mathcal{M}U = -\nabla p$. At that stage we enforce a zero-pressure gradient normal to the face by setting the pressure at the cell face equal to that of the center of the cell touching the slip wall. Therefore, it follows the same exact equation as Equation (36) and (39) giving up Equation (42).

$$\text{pressure correction flux @ slip wall face} \equiv (\mathcal{A}_f^{-1}) \frac{p'_f - p'_P}{d_{Pf}} S_f = 0 \quad (42)$$

Now for the velocity, the condition we are trying to enforce at a slip wall is that there is no velocity normal to the face. So, the component of velocity in the direction normal to the slip wall should equal zero. Mathematically that gives us Equation (43).

$$(U_{slip\ wall}^*) \cdot \mathbf{n}_{f,slip\ wall} = 0 \quad (43)$$

Simply plugging that into the intermediate velocity flux equation for a face would result in Equation (44).

$$\text{intermediate velocity flux @ slip wall face} \equiv (U_{slip\ wall}^*) \cdot \mathbf{n}_{f,slip\ wall} S_{f,slip\ wall} = 0 \quad (44)$$

This term is also zero. Therefore, for the slip wall, the flux contribution at the face for the pressure correction and intermediate velocity are both zero! That makes things easier because we don't have to care about it then. Nice!

Now following the pressure correction term is the actual application of the correction to the pressure and the velocity. Let us discuss the implementation of the SIMPLE Algorithm for FVM for the ...

Solution Update

Recall that we have to update the pressure and the velocity and we can do that via the previously derived Equation (17) and (18), which are shown again below so you don't have to scroll up.

$$p^{(k+1)} = p^{(k)} + \alpha_p p' \quad (17)$$

$$U^{(k+1)} = U^{(k)} + \alpha_U \left((U^* - \mathcal{A}^{-1} \nabla p') - U^{(k)} \right) \quad (18)$$

For the pressure update, it is literally just addition of vectors. We solved for p' in the previous step so just multiply the correction by the under relaxation factor then addition.

For the velocity update, we know all the terms and can easily evaluate them except for the pressure correction gradient term $\nabla p'$ at the cell center. We are going to evaluate this term by using a Green-Gauss gradient. The mathematical steps are shown below, but I just want to first emphasize that this approach assumes that the gradient is constant across the cell which allows us to use the face values. Maybe the math below will help explain that.

$$\int_A \nabla p' dA = \int_A \nabla \cdot (p' \mathbf{I}) dA = \oint_{dA} (p' \mathbf{I}) \cdot \mathbf{n} dA = \sum_f^{faces} (p'_f n_{f_x} + p'_f n_{f_y}) S_f \quad (45)$$

$$\int_A \nabla p' dA = \nabla p' \int_A dA = \nabla p' A \quad (46)$$

Setting Equation (45) and (46) equal to each other than rearranging allows us to recover Equation (47) to get the pressure correction gradient term.

$$\nabla p' = \frac{1}{A} \sum_f^{faces} (p'_f n_{fx} + p'_f n_{fy}) S_f \quad (47)$$

And, to be thorough, to get the value of the pressure correction at the face (since we are only storing the values at the cell center), we can just linearly interpolate as we've done before! The last thing we do need to consider before we can close the book is the ...

Boundary Condition

More so the point of this section is that we need the proper face values in order to evaluate the RHS of Equation (47) when we can't use linear interpolation at a boundary face (since there is no neighbor). I will make this brief, starting with the ...

Inlet

You can prescribe the pressure correction at the inlet face as the pressure correction at the inlet cell center. Zero-gradient pressure is what we want, and by making the correction to the center the same as the face, we maintain the zero-gradient.

Outlet

You can prescribe the pressure correction at the outlet face to zero. We already fix the value at the outlet face to the prescribed pressure outlet value. So, there is no need for a correction here.

Slip Wall

Similar to the inlet, we just want a zero-gradient pressure. We can apply the same philosophy and just make the pressure correction at the face equal to that of the cell center.

Conclusion

This is not really a conclusion or summary. This is just the end. We have all the terms to update the pressure and velocity as in Equation (17) and (18). That's the SIMPLE Algorithm implemented in FVM! And after we just need to repeat, repeat, repeat, until convergence (which is a whole other explanation which I am not going to talk about because this is already pretty long). That's it though! No conclusion, no summary, I will end it here. Goodbye and feel free to reach out if you had a question or something is suspicious (I would not put it past myself to make errors here and there)!